

Knapsack Problem

- ▶ A variation of a *bin packing* problem
- ▶ Similar to fair teams problem from recursion assignment
- ▶ You have a set of items
- ▶ Each item has a weight and a value
- ▶ You have a knapsack with a weight limit
- ▶ Goal: Maximize the **value** of the items you put in the knapsack without exceeding the weight limit

Knapsack Example

► Items:

Item Number	Weight of Item	Value of Item	Value per unit Weight
1	1	6	6.0
2	2	11	5.5
3	4	1	0.25
4	4	12	3.0
5	6	19	3.167
6	7	12	1.714

► Weight
Limit = 8

- One greedy solution: Take the highest ratio item that will fit: (1, 6), (2, 11), and (4, 12)
- Total value = $6 + 11 + 12 = 29$
- Clicker 3 - Is this optimal? A. No B. Yes

Knapsack - Recursive Backtracking

```
private static int knapsack(ArrayList<Item> items,
    int current, int capacity) {

    int result = 0;
    if (current < items.size()) {
        // don't use item
        int withoutItem
            = knapsack(items, current + 1, capacity);
        int withItem = 0;
        // if current item will fit, try it
        Item currentItem = items.get(current);
        if (currentItem.weight <= capacity) {
            withItem += currentItem.value;
            withItem += knapsack(items, current + 1,
                capacity - currentItem.weight);
        }
        result = Math.max(withoutItem, withItem);
    }
    return result;
}
```

Knapsack - Dynamic Programming

- ▶ Recursive backtracking starts with max capacity and makes choice for items: choices are:
 - take the item if it fits
 - don't take the item
- ▶ Dynamic Programming, start with simpler problems
- ▶ Reduce number of items available
- ▶ ... AND Reduce weight limit on knapsack
- ▶ Creates a 2d array of possibilities

Knapsack - Optimal Function

- ▶ `OptimalSolution(items, weight)` is best solution given a subset of items and a weight limit
- ▶ 2 options:
- ▶ `OptimalSolution` does not select i^{th} item
 - select best solution for items 1 to $i - 1$ with weight limit of w
- ▶ `OptimalSolution` selects i^{th} item
 - New weight limit = $w - \text{weight of } i^{\text{th}} \text{ item}$
 - select best solution for items 1 to $i - 1$ with new weight limit

Knapsack Optimal Function

► $\text{OptimalSolution}(\text{items}, \text{weight limit}) =$

0 if 0 items

$\text{OptimalSolution}(\text{items} - 1, \text{weight})$ if weight of i^{th} item is greater than allowed weight
 $w_i > w$ (In others i^{th} item doesn't fit)

max of ($\text{OptimalSolution}(\text{items} - 1, w)$,
value of i^{th} item +
 $\text{OptimalSolution}(\text{items} - 1, w - w_i)$)

Knapsack - Algorithm

- ▶ Create a 2d array to store value of best option given subset of items and possible weights

Item Number	Weight of Item	Value of Item
1	1	6
2	2	11
3	4	1
4	4	12
5	6	19
6	7	12

- ▶ In our example 0 to 6 items and weight limits of 0 to 8
- ▶ Fill in table using OptimalSolution Function

Knapsack Algorithm

Given N items and WeightLimit

Create Matrix M with N + 1 rows and WeightLimit + 1 columns

For weight = 0 to WeightLimit

$M[0, w] = 0$

For item = 1 to N

for weight = 1 to WeightLimit

if(weight of ith item > weight)

$M[\text{item}, \text{weight}] = M[\text{item} - 1, \text{weight}]$

else

$M[\text{item}, \text{weight}] = \max \text{ of}$

$M[\text{item} - 1, \text{weight}] \text{ AND}$

value of item + $M[\text{item} - 1, \text{weight} - \text{weight of item}]$

Knapsack - Completed Table

items / weight	0	1	2	3	4	5	6	7	8
{}	0	0	0	0	0	0	0	0	0
{1} [1, 6]	0	6	6	6	6	6	6	6	6
{1,2} [2, 11]	0	6	11	17	17	17	17	17	17
{1, 2, 3} [4, 1]	0	6	11	17	17	17	17	18	18
{1, 2, 3, 4} [4, 12]	0	6	11	17	17	18	23	29	29
{1, 2, 3, 4, 5} [6, 19]	0	6	11	17	17	18	23	29	30
{1, 2, 3, 4, 5, 6} [7, 12]	0	6	11	17	17	18	23	29	30

Knapsack - Items to Take

items / weight	0	1	2	3	4	5	6	7	8
{}	0	0	0	0	0	0	0	0	0
{1} [1, 6]	0	6	6	6	6	6	6	6	6
{1,2} [2, 11]	0	6	11	17	17	17	17	17	17
{1, 2, 3} [4, 1]	0	6	11	17	17	17	17	17	17
{1, 2, 3, 4} [4, 12]	0	6	11	17	17	18	23	29	29
{1, 2, 3, 4, 5} [6, 19]	0	6	11	17	17	18	23	29	30
{1, 2, 3, 4, 5, 6} [7, 12]	0	6	11	17	17	18	23	29	30